Dynamic Programming

Reinforcement Learning · Lecture 2

Claire Vernade
University of Technology Nuremberg

22 October 2026

Today

1. **Recap & notation** — and our running example: the two-exit maze
2. **Value functions** — how good is a policy?
3. **Bellman equations** — expectation, optimality, operators
4. **Dynamic programming** — policy evaluation, policy iteration, value iteration

TODAY'S ASSUMPTION

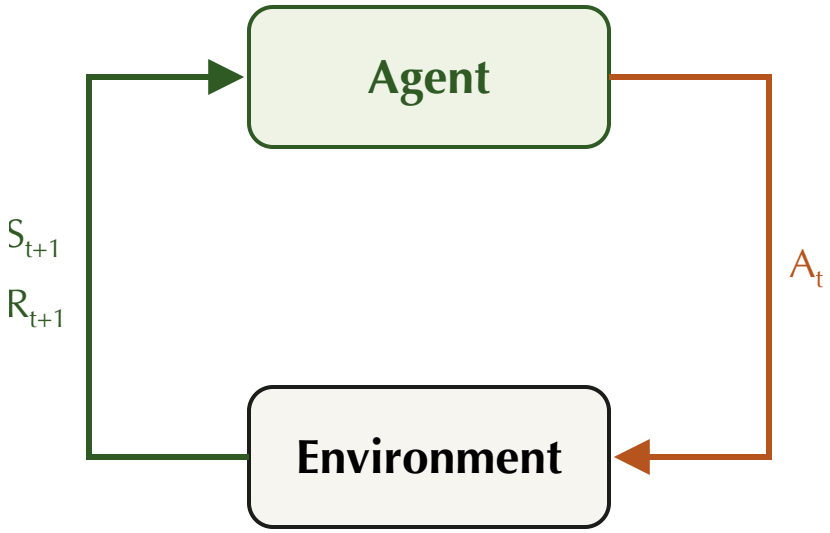
The MDP is **known**: we can query P and r.
This is **planning**, not yet learning.

PART 1

Recap & running example

Notation, the maze we will use all lecture, and the objective.

Agent and environment



At each time step $t = 0, 1, 2, \ldots$

- the agent observes the state $S_t \in \mathcal{S}$
- picks an action $A_t \in \mathcal{A}$
- receives a reward R_{t+1} and moves to S_{t+1}

Trajectory: $S_0, A_0, R_1, S_1, A_1, R_2, S_2, \ldots$

RECAP

Markov Decision Process

DEFINITION

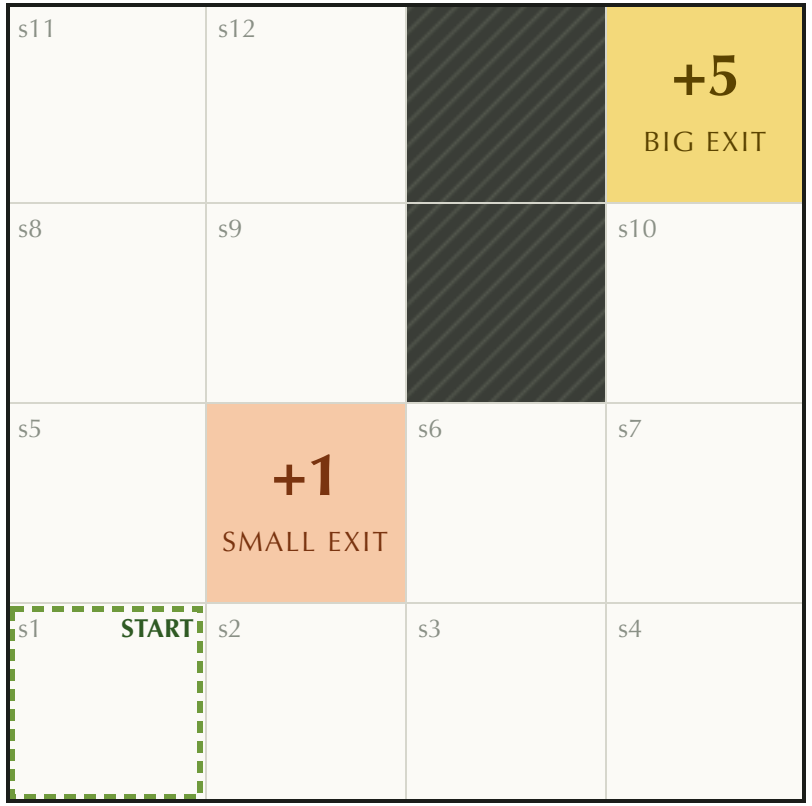
A (discounted) MDP is a tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma)$:

- $\mathcal{S}$ finite set of states, $\mathcal{A}$ finite set of actions
- $P(s' \mid s, a)$ transition probabilities — *Markov*: depends only on (s, a)
- $r(s, a)$ expected immediate reward, $r(s, a) = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$
- $\gamma \in (0, 1)$ discount factor

An MDP is a **model** of the problem. It assumes the state is fully observed, the dynamics are Markov, and the goal can be written as a cumulative reward (the "reward hypothesis").

RUNNING EXAMPLE

The two-exit maze



- 12 states $s_1, \ldots, s_{12}$, start at s_1
- Actions: $\mathcal{A} = \{\uparrow, \rightarrow, \downarrow, \leftarrow\}$
- **Deterministic** moves; walls and borders: stay put
- Two exits end the episode:
  - **small exit**: $+1$, close by
  - **big exit**: $+5$, far away
- All other rewards are 0

DISCUSS WITH YOUR NEIGHBOUR

Which exit should the agent go to?

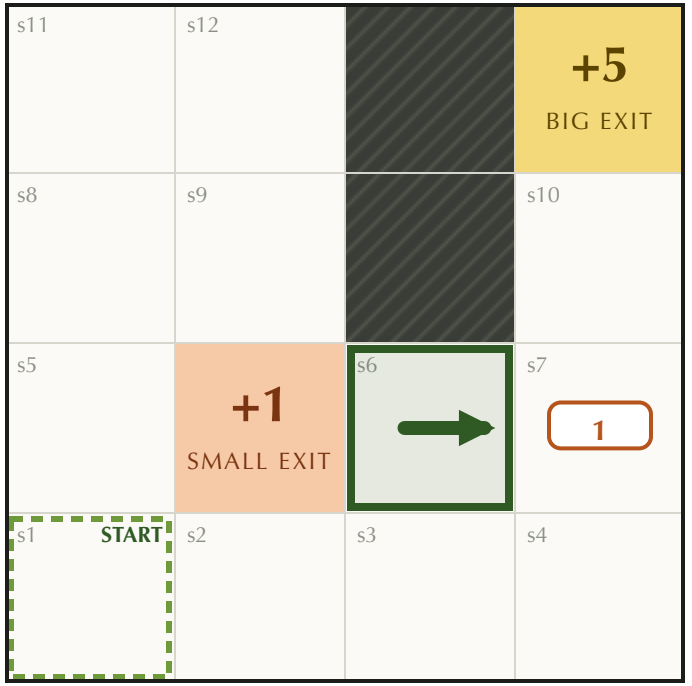
RUNNING EXAMPLE

The maze as an MDP

- $\mathcal{S} = \{s_1, \dots, s_{12}\}$ + two **terminal** exits. $r(s, a) = 5$ or 1 when a enters an exit, else 0
- Deterministic: $P(s' \mid s, a) \in \{0, 1\}$, e.g. $P(s_7 \mid s_6, \rightarrow) = 1$, $P(s_1 \mid s_1, \leftarrow) = 1$

> **VARIANT: THE SLIPPERY MAZE**
>
> W.p. 0.8 the intended move; w.p. 0.2 a **slip** to one of the 3 other neighbours ($0.2/3$ each). Slipping into a wall = staying put.

- **Stochastic transitions**: same state, same action, several possible next states
- Model **external perturbations**: wind, slippery floor, actuator noise, other agents
- $r(s, a)$ is now an expectation: $r(s_6, \rightarrow) = \frac{0.2}{3} \cdot 1 \approx 0.07$



$P(\cdot \mid s_6, \rightarrow)$. Toggle to compare.

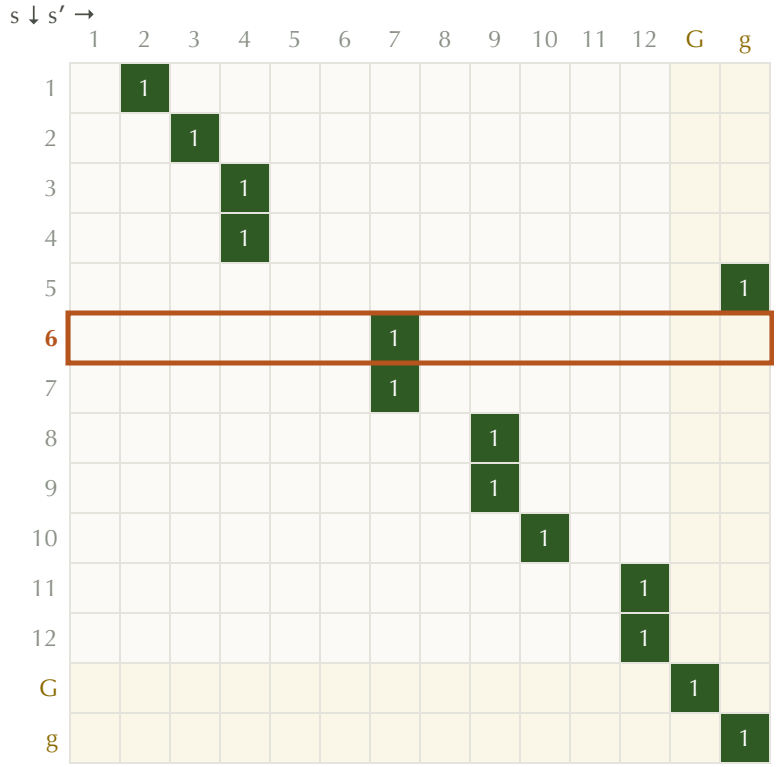
In the computer: P and r are arrays

```
S, A = 14, 4   # 12 cells + 2 exits; ↑ → ↓ ←
P = np.zeros((A, S, S))  # P[a, s, s']
r = np.zeros((S, A))     # r[s, a]

# every row is a distribution
assert np.allclose(P.sum(axis=2), 1)

# one Bellman backup, all (s, a) at once
Q = r + gamma * np.einsum("ast,t->sa", P, V)
V = Q.max(axis=1)
```

- $P = |\mathcal{A}| = 4$ **stochastic matrices** of size $|\mathcal{S}| \times |\mathcal{S}|$
- Walls are not states. The exits are **absorbing**: $P(G \mid G, a) = 1, r = 0$
- Memory $|\mathcal{A}|\,|\mathcal{S}|^2$: 784 numbers here; impossible for chess or Go

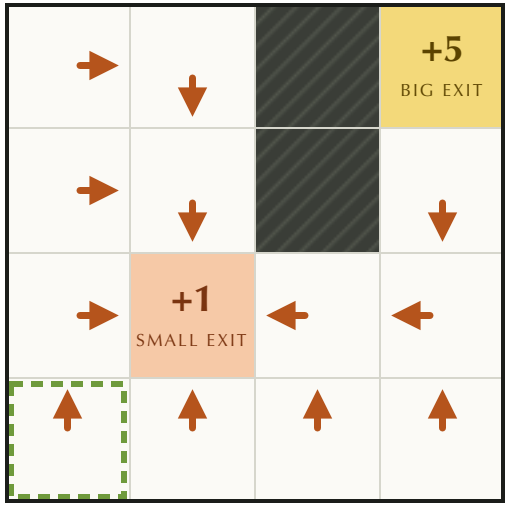


$P[\rightarrow] \in [0,1]^{14\times14}$ · 14 non-zeros · row s6 sums to 1.00

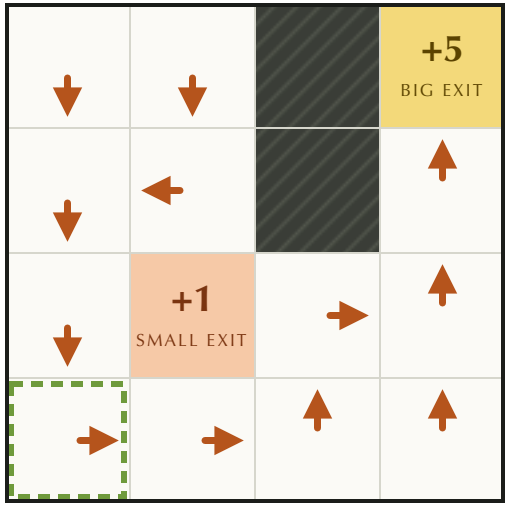
Policies

DEFINITION

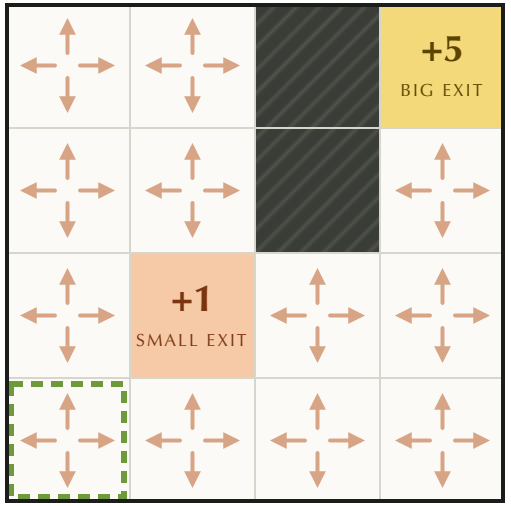
A (stationary, Markov) **policy** maps states to distributions over actions: $\pi(a \mid s) = \mathbb{P}(A_t = a \mid S_t = s)$.
Deterministic policy: $\pi : \mathcal{S} \to \mathcal{A}$.



π_{small}: shortest path to $+1$



π_{big}: shortest path to $+5$



π_{unif}: uniformly random

The return

DEFINITION

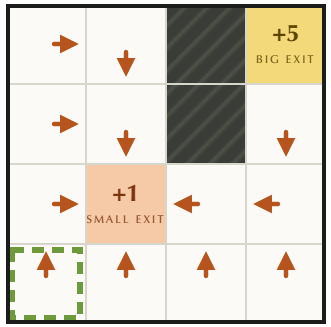
The (discounted) return from time t:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Following π_{small} from s_1

$s_1 \xrightarrow{\uparrow} s_5 \xrightarrow{\rightarrow}$ small exit

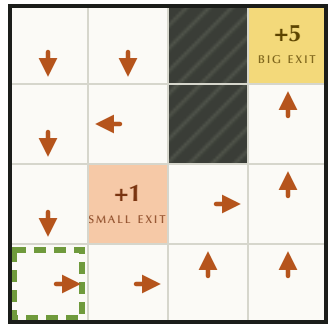
$$G_0 = 0 + \gamma \cdot 1 = \gamma$$



Following π_{big} from s_1

$s_1 \to s_2 \to s_3 \to s_6 \to s_7 \to s_{10} \to$ big exit

$$G_0 = 0 + \cdots + 0 + \gamma^5 \cdot 5$$



Let's practice: when is the big exit worth it?

From s_1, compare γ (small exit) and $5\gamma^5$ (big exit).

$$5\gamma^5 > \gamma \iff \gamma^4 > \tfrac{1}{5} \iff \gamma > 5^{-1/4} \approx 0.669$$

- $\gamma = 0.9$: $5 \cdot 0.9^5 \approx 2.95 > 0.9$ → go **big**
- $\gamma = 0.5$: $5 \cdot 0.5^5 \approx 0.16 < 0.5$ → go **small**

THINK

Is the threshold the same for every starting state? (Try s_6, next to both routes.)

Why discount?

- Keeps returns **finite**: if $|R_t| \leq R_{\max}$ then $|G_t| \leq \frac{R_{\max}}{1-\gamma}$
- Expresses **uncertainty about the future** ($\rightarrow$)
- **Effective horizon** $\mathbb{E}[T] = \frac{1}{1-\gamma}$: $\gamma = 0.9 \Rightarrow 10$ steps, $\gamma = 0.99 \Rightarrow 100$
- Humans and animals prefer immediate rewards

TAKE-AWAY

γ is part of the **problem**, not of the algorithm. Changing γ changes what "optimal" means.

DISCOUNTING = A GEOMETRIC BELIEF ABOUT THE END

Suppose the episode ends after each step w.p. $1-\gamma$: a random horizon T with $\mathbb{P}(T > k) = \gamma^k$. The expected **undiscounted** reward until T is

$$\mathbb{E}_T\left[\sum_{k=0}^{T-1} R_{t+k+1}\right] = \sum_{k\geq 0} \mathbb{P}(T > k)\, R_{t+k+1} = \sum_{k\geq 0} \gamma^k R_{t+k+1} = G_t$$

A reward k steps ahead counts with weight γ^k = the probability of still being there.

BEYOND GEOMETRIC

If we are uncertain about the stopping rate itself, the weights become **hyperbolic**, $\propto 1/(1+\kappa k)$ (Fedus et al., *Hyperbolic discounting and learning over multiple horizons*, arXiv:1902.06865). This raises additional computational questions (no single Bellman recursion); out of scope here.

The objective

GOAL

Find a policy π maximising the **expected discounted return**

$$\max_{\pi} \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \,\middle|\, S_0 = s\right] \quad \text{for every } s$$

Prediction

Given π: how good is it? → **evaluate** π

Control

Find the best π → **optimise**

When the MDP is known, this is an **optimisation** problem (planning). When it is unknown, it's a **learning** problem (next lectures).

PART 2

Value functions

How good is it to be in a state, or to take an action, when following π?

State-value function

DEFINITION

The **value** of state s under policy π is the expected return when starting in s and following π:

$$V^\pi(s) = \mathbb{E}_\pi\left[G_t \mid S_t = s \right]$$

- $\mathbb{E}_\pi$: actions drawn from π, next states from P
- A value function is just a **vector** $V^\pi \in \mathbb{R}^{|\mathcal{S}|}$, one number per state. In the maze, 12 numbers.

Action-value function

DEFINITION

First take action a in s, then follow π:

$$Q^\pi(s,a) = \mathbb{E}_\pi\big[\, G_t \mid S_t = s, A_t = a \,\big]$$

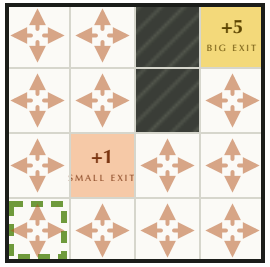
The two are linked:

$$V^\pi(s) = \sum_a \pi(a \mid s)\, Q^\pi(s,a) \qquad\qquad Q^\pi(s,a) = r(s,a) + \gamma \sum_{s'} P(s' \mid s,a)\, V^\pi(s')$$

MAZE

V^π of the uniformly random policy, $\gamma = 0.9$

s11 0.32	s12 0.36		+5 BIG EXIT
s8 0.43	s9 0.55		s10 2.81
s5 0.60	+1 SMALL EXIT	s6 0.91	s7 1.32
s1 START 0.52	s2 0.68	s3 0.70	s4 0.83



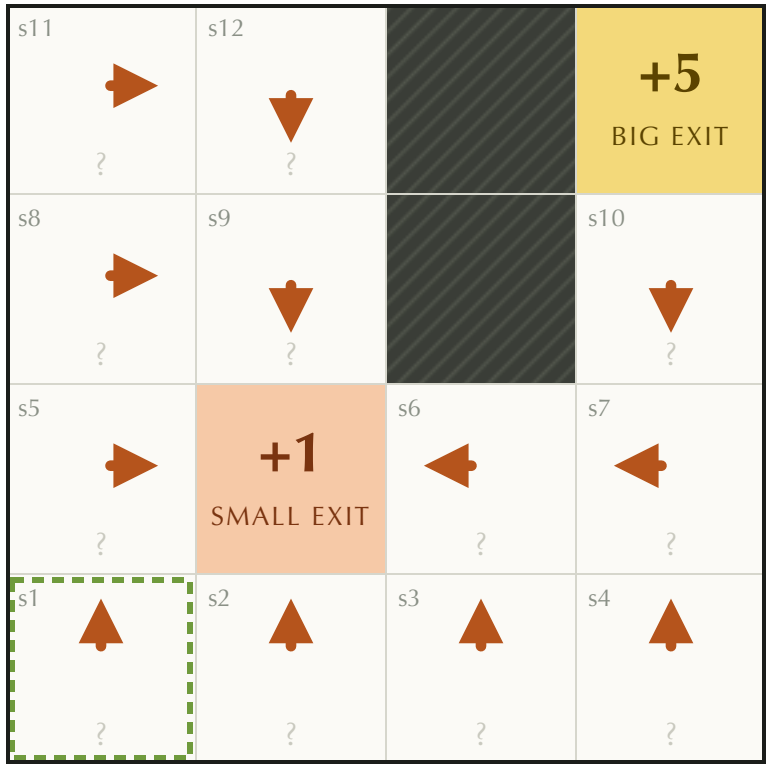
Fix π = **uniformly random**: each action w.p. $\frac{1}{4}$.
I computed V^π for you (left).

- Why is s_{10} the most valuable state?
- Why is $V(s_5) > V(s_1)$?
- The random walker often bumps into walls; it can still stumble into the big exit.

OPEN QUESTIONS

- How did I compute these numbers? (Parts 3 and 4)
- Can I **improve** the policy using this function? (Part 4)

Let's practice: value of π_{small}



$\gamma = 0.9$. Policy is deterministic, moves are deterministic: the return is a **single number**.

If the exit is d steps away: $V^\pi(s) = \gamma^{d-1} \cdot 1$.

Click a cell to reveal its value.

THINK

Do you need to compute each value independently of all the others, already computed ones?

No: e.g. $V^\pi(s_1) = 0 + \gamma\, V^\pi(s_5) = 0.9 \cdot 1$. Each value is the reward plus $\gamma\times$ the value of the next state. That is the **Bellman equation**.

PART 3

Bellman equations

Value = immediate reward + discounted value of where you land.

DERIVATION 1/2

Bellman expectation equation

Recall the return: $G_t = \sum_{k \geq 0} \gamma^k R_{t+k+1} = R_{t+1} + \gamma\big(R_{t+2} + \gamma R_{t+3} + \dots\big) = R_{t+1} + \gamma\, G_{t+1}$

Fix a policy π (e.g. π_{big}). Goal: compute $V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$ for all s, **efficiently**.

$$V^\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma\, G_{t+1} \mid S_t = s] = \underbrace{\mathbb{E}_\pi[R_{t+1} \mid S_t = s]}_{\text{immediate reward}} + \gamma\, \underbrace{\mathbb{E}_\pi[G_{t+1} \mid S_t = s]}_{\text{future}}$$

WHY THE FUTURE TERM IS $\mathbb{E}_\pi[V^\pi(S_{t+1}) \mid S_t = s]$

Condition on where we land, then use the **Markov property**: once $S_{t+1} = s'$ is known, what happens next does not depend on the past.

$$\mathbb{E}_\pi[G_{t+1} \mid S_t = s] = \mathbb{E}_\pi\big[\,\mathbb{E}_\pi[G_{t+1} \mid S_{t+1}] \,\big|\, S_t = s\big] = \mathbb{E}_\pi[V^\pi(S_{t+1}) \mid S_t = s]$$

Careful: $G_{t+1} \neq V^\pi(S_{t+1})$ as random variables. They are only equal **in expectation** given S_{t+1} (tower rule).

DERIVATION 2/2

Bellman expectation equation

Develop each term: first draw $a \sim \pi(\cdot \mid s)$, then $s' \sim P(\cdot \mid s, a)$.

Immediate reward

$$\mathbb{E}_\pi[R_{t+1} \mid S_t = s] = \sum_a \pi(a \mid s)\, r(s, a)$$

Future

$$\mathbb{E}_\pi[V^\pi(S_{t+1}) \mid S_t = s] = \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a)\, V^\pi(s')$$

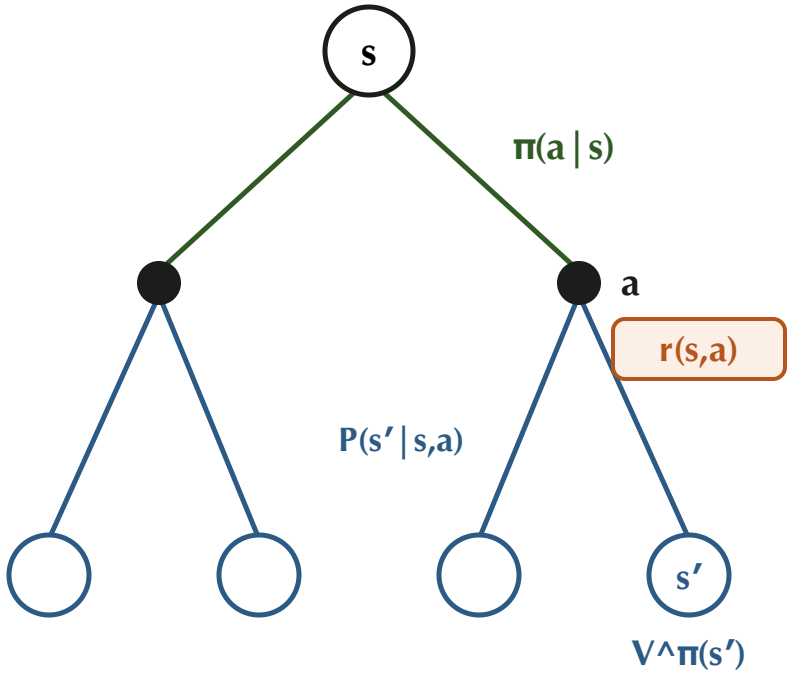
RECOMBINE: BELLMAN EXPECTATION EQUATION

$$V^\pi(s) = \sum_a \pi(a \mid s) \Big(r(s, a) + \gamma \sum_{s'} P(s' \mid s, a)\, V^\pi(s') \Big)$$

One equation per state: $|\mathcal{S}|$ **linear** equations in the $|\mathcal{S}|$ unknowns $V^\pi(s)$.

Same for Q: $Q^\pi(s, a) = r(s, a) + \gamma \sum_{s'} P(s' \mid s, a) \sum_{a'} \pi(a' \mid s')\, Q^\pi(s', a')$

Backup diagram



$$V^\pi(s) = \sum_a \pi(a \mid s) \left(r(s,a) + \gamma \sum_{s'} P(s' \mid s,a)\, V^\pi(s') \right)$$

- open circle = state, filled = action
- **top level**: average over actions with π
- **at the action**: collect the reward $r(s,a)$
- **bottom level**: average over next states with P, weighting their values $V^\pi(s')$
- "back up" the leaves to the root: that is one Bellman update

Let's practice: Bellman equation in the maze



Uniformly random policy, $\gamma = 0.9$. Write the equation for s_{10}:

$$V^\pi(s_{10}) = \tfrac{1}{4}\left[(5+0) + \gamma V^\pi(s_{10}) + \gamma V^\pi(s_7) + \gamma V^\pi(s_{10})\right]$$

↑ exit, → wall, ↓ s_7, ← wall. Click any state to check.

Click a state to see its Bellman backup.

Matrix form and closed-form solution

Stack everything into vectors: $r^\pi(s) = \sum_a \pi(a|s) r(s,a)$, $P^\pi(s,s') = \sum_a \pi(a|s) P(s'|s,a)$.

$$V^\pi = r^\pi + \gamma P^\pi V^\pi \quad \Longrightarrow \quad V^\pi = (I - \gamma P^\pi)^{-1} r^\pi$$

- A **linear** system: $|\mathcal{S}|$ equations, $|\mathcal{S}|$ unknowns
- $I - \gamma P^\pi$ is invertible because $\gamma < 1$ and P^π is stochastic
- Cost $O(|\mathcal{S}|^3)$: fine for 12 states, not for 10^6 states → iterative methods

Bellman operator

DEFINITION

For any $V \in \mathbb{R}^{|\mathcal{S}|}$, define $\mathcal{T}^\pi V \in \mathbb{R}^{|\mathcal{S}|}$ by

$$(\mathcal{T}^\pi V)(s) = \sum_a \pi(a \mid s)\Big(r(s,a) + \gamma \sum_{s'} P(s' \mid s,a)\, V(s')\Big)$$

The Bellman equation says: V^π is a **fixed point**, $\mathcal{T}^\pi V^\pi = V^\pi$.

PROPOSITION (CONTRACTION)

$\|\mathcal{T}^\pi U - \mathcal{T}^\pi V\|_\infty \leq \gamma \|U - V\|_\infty$ for all U, V.
By Banach's fixed-point theorem, V^π is the **unique** fixed point, and $V_{k+1} = \mathcal{T}^\pi V_k$ converges to it from any V_0.

Optimal value functions

DEFINITION

$$V^\star(s) = \max_\pi V^\pi(s), \qquad Q^\star(s,a) = \max_\pi Q^\pi(s,a)$$

A policy $\pi^\star$ is **optimal** if $V^{\pi^\star}(s) = V^\star(s)$ for **all** s.

DISCUSS

$V^\star(s)$ is a maximum taken **separately for each state**. Could the best policy when starting from s_1 differ from the best one when starting from s_6? Is there a single policy that is best **everywhere at once**?

Not obvious a priori. For MDPs the answer is yes: see the next slide.

Bellman optimality equation

$$V^\star(s) = \max_a \left(r(s,a) + \gamma \sum_{s'} P(s' \mid s,a)\, V^\star(s') \right) = \max_a Q^\star(s,a)$$

OPTIMAL BELLMAN OPERATOR

$(\mathcal{T}^\star V)(s) = \max_a \left(r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V(s') \right)$ — also a γ-contraction.

- **Non-linear** (because of the max): no closed form in general
- Solve iteratively: value iteration, policy iteration (today), Q-learning, SARSA (Chris, in two weeks)

Fundamental theorem of MDPs

THEOREM

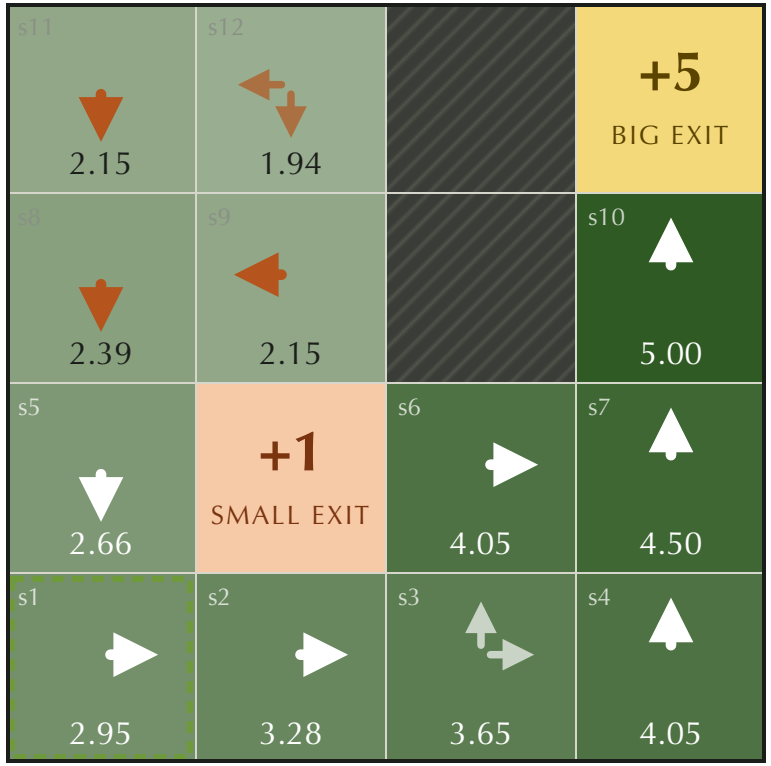
For a finite discounted MDP:

1. There is a unique $V^\star$ with $\mathcal{T}^\star V^\star = V^\star$.
2. There exists an optimal policy that is **stationary and deterministic**.
3. A policy is optimal **iff** it is greedy w.r.t. $Q^\star$: $\pi^\star(s) \in \arg\max_a Q^\star(s, a)$.

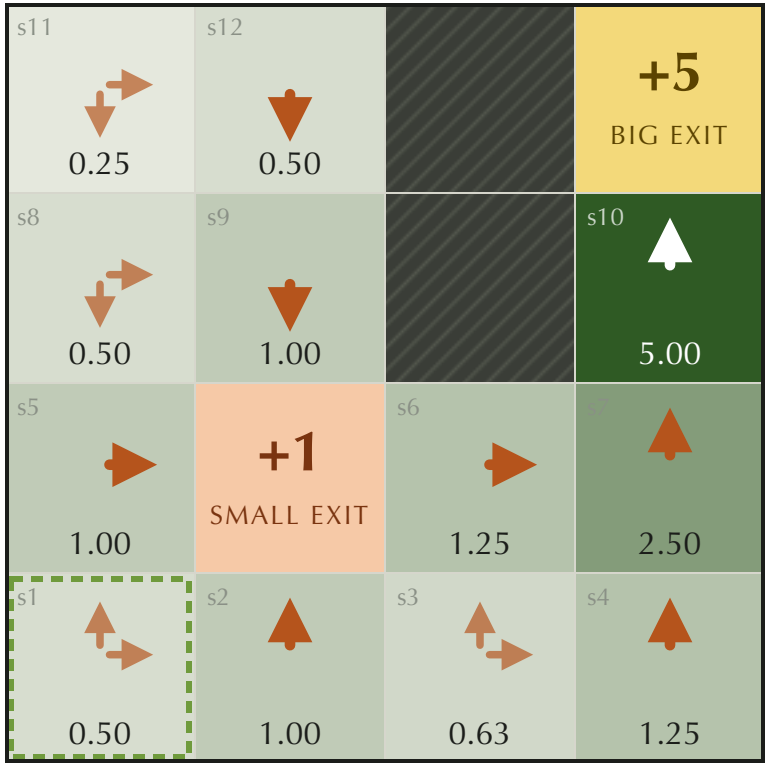
So we only need to compute $V^\star$ (or $Q^\star$); acting greedily then gives an optimal policy.

MAZE

$V^\star$ and $\pi^\star$ depend on γ



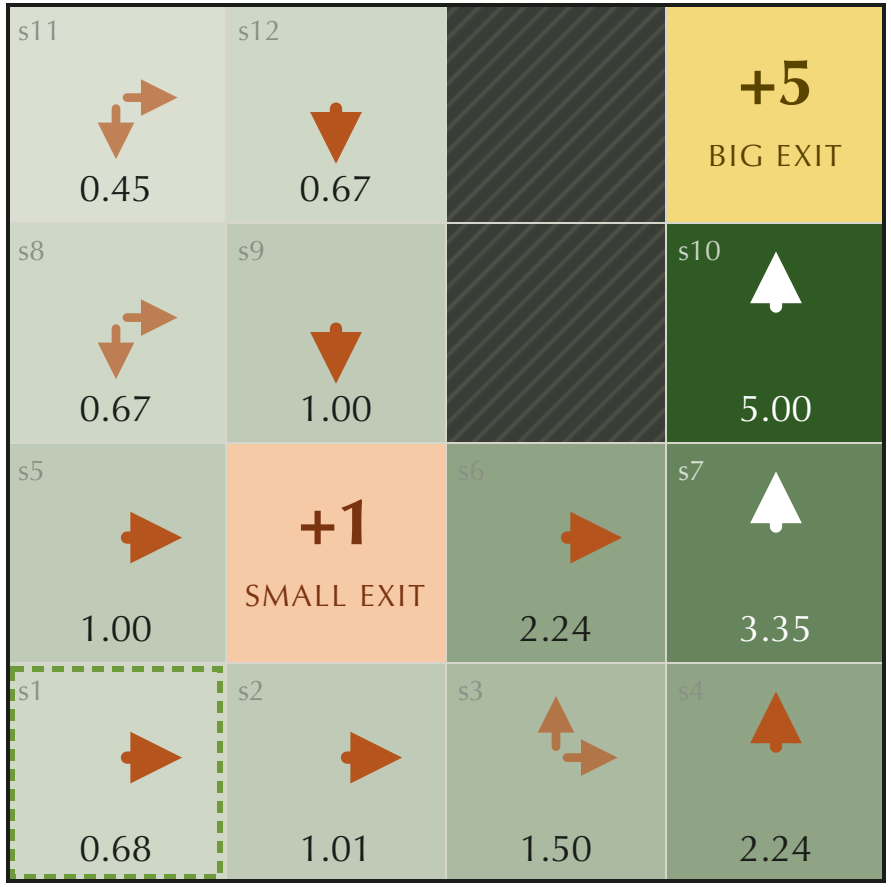
$\gamma = 0.9$



$\gamma = 0.5$

MAZE

Play with γ



start state: $V(s_1) = 0.675$

Click a state to see its Bellman backup.

Questions?

5-minute break

PART 4

Dynamic programming

Turn the Bellman equations into algorithms.

What is dynamic programming?

- **Dynamic**: sequential problem. **Programming**: optimising a "program", i.e. a policy (Bellman, 1957)
- Applies to problems with
  - **optimal substructure**: an optimal solution is built from optimal solutions to subproblems
  - **overlapping subproblems**: solutions can be cached and reused
- MDPs have both: Bellman equations give the recursion, and the value function is the cache

REQUIRES

Full knowledge of the MDP: P and r.

Policy evaluation (prediction)

Problem: given π, compute V^π. **Idea**: iterate the Bellman operator.

ITERATIVE POLICY EVALUATION

1. Initialise $V_0(s) = 0$ for all s
2. For $k = 0, 1, 2, \ldots$: $V_{k+1} = \mathcal{T}^\pi V_k$, i.e. for every s,

$$V_{k+1}(s) = \sum_a \pi(a|s) \Big(r(s,a) + \gamma \sum_{s'} P(s'|s,a)\, V_k(s') \Big)$$

3. Stop when $\|V_{k+1} - V_k\|_\infty$ is small

Convergence: $\|V_k - V^\pi\|_\infty \leq \gamma^k \, \|V_0 - V^\pi\|_\infty$, by contraction.

Let's practice: policy evaluation by hand



$k = 0$

π_{unif}, $\gamma = 0.9$, $V_0 = 0$.

1. Compute V_1: which states change? (Only neighbours of an exit.)
2. Compute $V_2(s_7)$ and $V_2(s_{10})$ by hand.
3. Press **Step** to check; click a state for its backup.

Click a state to see its Bellman backup.

Policy improvement

Given V^π, act **greedily**:

$$\pi'(s) \in \arg\max_a Q^\pi(s,a) = \arg\max_a \left(r(s,a) + \gamma \sum_{s'} P(s'|s,a)\, V^\pi(s') \right)$$

POLICY IMPROVEMENT THEOREM

If $Q^\pi(s, \pi'(s)) \geq V^\pi(s)$ for all s, then $V^{\pi'}(s) \geq V^\pi(s)$ for all s.

If the improvement stops ($\pi' = \pi$), then V^π satisfies the Bellman **optimality** equation, so π is optimal.

Proof: see the **self-study** section at the end.

Policy iteration

POLICY ITERATION

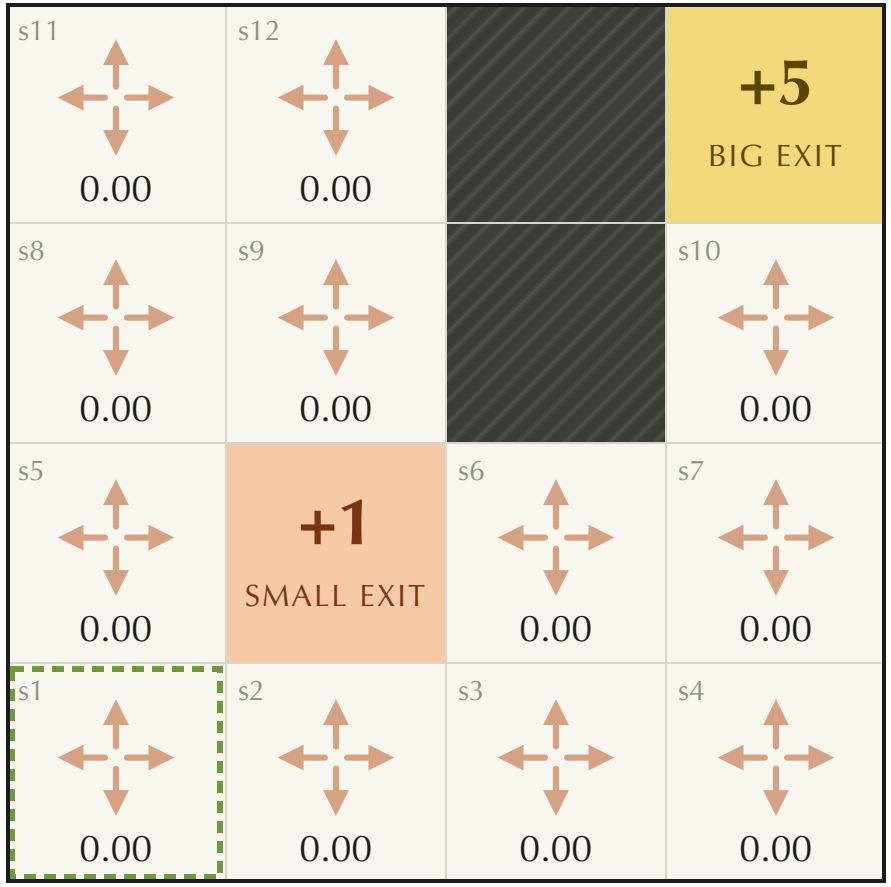
1. Start from any policy π_0
2. **Evaluate**: compute V^{π_k} (exactly, or by iterative policy evaluation)
3. **Improve**: $\pi_{k+1} = \mathrm{greedy}(V^{\pi_k})$
4. Repeat until $\pi_{k+1} = \pi_k$

$$\pi_0 \xrightarrow{E} V^{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} V^{\pi_1} \xrightarrow{I} \cdots \rightarrow \pi^\star$$

Terminates in finitely many steps: there are finitely many deterministic policies, and each step strictly improves until optimal.

MAZE

Policy iteration from the random policy



iteration 0 · next: **evaluate**

Value iteration

Idea: skip full evaluation and iterate the **optimal** Bellman operator directly.

VALUE ITERATION

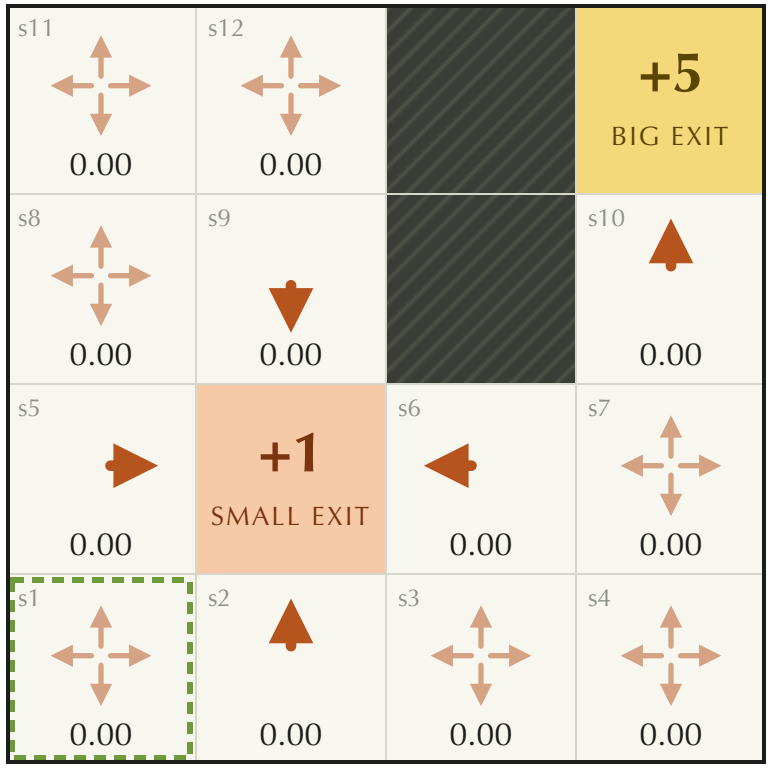
1. Initialise $V_0(s) = 0$ for all s
2. For $k = 0, 1, 2, \ldots$: $V_{k+1} = \mathcal{T}^\star V_k$, i.e.

$$V_{k+1}(s) = \max_a \left(r(s,a) + \gamma \sum_{s'} P(s'|s,a)\, V_k(s') \right)$$

3. Return the greedy policy w.r.t. the final V_k

Converges to $V^\star$ at rate γ^k. You can view it as policy iteration with one sweep of evaluation per improvement.

Let's practice: value iteration by hand



$k = 0$

$\gamma = 0.9$, $V_0 = 0$. Compute V_1, V_2, V_3.

- V_1: 1 next to the small exit, 5 at s_{10}, 0 elsewhere
- $V_2(s_7) = \max(\ldots, 0 + 0.9 \cdot 5) = 4.5$
- When does s_1 first "see" the big exit? After how many iterations is its greedy action final?

Click a state to see its Bellman backup.

Value iteration vs policy iteration

	Value iteration	Policy iteration
Per iteration	one sweep of $\mathcal{T}^\star$: $O(\lvert\mathcal{S}\rvert^2\lvert\mathcal{A}\rvert)$	full evaluation $O(\lvert\mathcal{S}\rvert^3)$ + improvement
Iterations	many (rate γ^k)	few (finite, often very few)
Intermediate V_k	not the value of any policy	always V^{π_k}

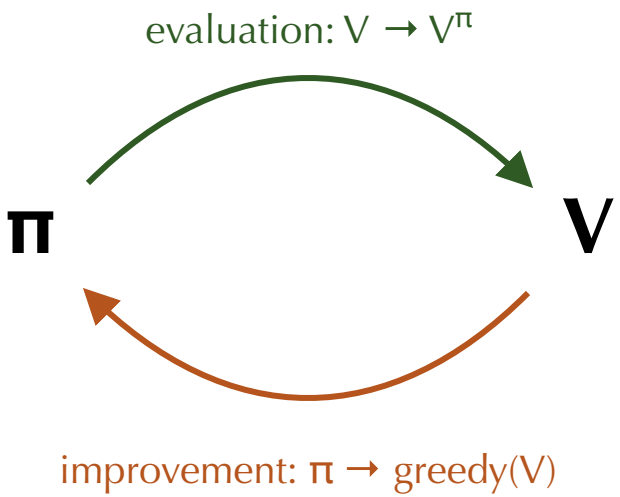
STOPPING RULE FOR VI

If $\|V_{k+1} - V_k\|_\infty < \varepsilon \frac{1-\gamma}{2\gamma}$, the greedy policy w.r.t. V_{k+1} is ε-optimal.

Puterman (1994), Thm 6.3.1. Proof and discussion: **self-study** section at the end.

OPTIONAL

Generalised policy iteration



- Any interleaving of (partial) evaluation and (partial) improvement
- VI and PI are the two extremes
- Almost every RL algorithm in this course is GPI with **approximate** evaluation: MC, TD, SARSA, Q-learning, actor-critic

Summary

Problem	Bellman equation	Algorithm
Prediction	Expectation: $V^\pi = \mathcal{T}^\pi V^\pi$	Iterative policy evaluation
Control	Expectation + greedy improvement	Policy iteration
Control	Optimality: $V^\star = \mathcal{T}^\star V^\star$	Value iteration

- All methods here are **synchronous**: every state is updated in each sweep. Asynchronous variants update states in any order.
- They all need the model and touch every state, which breaks down for large $\mathcal{S}$ and unknown P.

Next time: Model-free prediction

What if we **don't know** P and r, and can only **sample** trajectories in the maze?

- Monte Carlo: average sampled returns
- Temporal differences: bootstrap from $\gamma V(S_{t+1})$

Same maze, same γ. We will compare with the exact values computed today.

SELF-STUDY

Proofs

Not covered in class. Everything follows from two properties of the Bellman operators: monotonicity and contraction.

SELF-STUDY

Two tools

Inequalities between value functions are pointwise: $U \leq V$ means $U(s) \leq V(s)$ for all s.

LEMMA 1: MONOTONICITY

If $U \leq V$, then $\mathcal{T}^\pi U \leq \mathcal{T}^\pi V$ and $\mathcal{T}^\star U \leq \mathcal{T}^\star V$.

Proof: $(\mathcal{T}^\pi V)(s) = \sum_a \pi(a|s)\big(r(s,a) + \gamma \sum_{s'} P(s'|s,a)V(s')\big)$ only adds up V with non-negative weights. For $\mathcal{T}^\star$, a maximum of larger terms is larger.

LEMMA 2: CONTRACTION (FROM THE LECTURE)

$\|\mathcal{T}^\pi U - \mathcal{T}^\pi V\|_\infty \leq \gamma \|U - V\|_\infty$, and the same for $\mathcal{T}^\star$. Hence $(\mathcal{T}^\pi)^k V \to V^\pi$ and $(\mathcal{T}^\star)^k V \to V^\star$ for any V.

For $\mathcal{T}^\star$, use $|\max_a f(a) - \max_a g(a)| \leq \max_a |f(a) - g(a)|$.

GREEDY POLICIES

If π' is greedy w.r.t. V, i.e. $\pi'(s) \in \arg\max_a \big(r(s,a) + \gamma \sum_{s'} P(s'|s,a)V(s')\big)$, then

$$T^{\pi'} V = \mathcal{T}^\star V \;\geq\; \mathcal{T}^\pi V \quad \text{for every policy } \pi.$$

SELF-STUDY

Proof of the policy improvement theorem

Let π' be greedy w.r.t. V^π. (More generally, assume $Q^\pi(s, \pi'(s)) \geq V^\pi(s)$ for all s, i.e. $T^{\pi'} V^\pi \geq V^\pi$.)

$$T^{\pi'} V^\pi \;=\; \mathcal{T}^\star V^\pi \;\geq\; \mathcal{T}^\pi V^\pi \;=\; V^\pi$$

Apply $T^{\pi'}$ on both sides and use monotonicity (Lemma 1), again and again:

$$V^\pi \;\leq\; T^{\pi'} V^\pi \;\leq\; (T^{\pi'})^2 V^\pi \;\leq\; \cdots \;\leq\; (T^{\pi'})^k V^\pi \xrightarrow[k \to \infty]{} V^{\pi'} \quad \text{(Lemma 2)}$$

Hence $V^{\pi'} \geq V^\pi$. ■

SAME PROOF, UNROLLED ("TELESCOPING", AS IN SUTTON & BARTO §4.2)

$V^\pi(s) \leq Q^\pi(s, \pi'(s)) = \mathbb{E}_{\pi'}[R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s] \leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 V^\pi(S_{t+2}) \mid S_t = s] \leq \cdots \leq V^{\pi'}(s)$

SELF-STUDY

Policy iteration ends with an optimal policy

Let π_{k+1} be greedy w.r.t. V^{π_k}. By the improvement theorem, $V^{\pi_{k+1}} \geq V^{\pi_k}$.

IF THERE IS NO IMPROVEMENT, THE POLICY IS OPTIMAL

Suppose $V^{\pi_{k+1}} = V^{\pi_k}$. Then

$$V^{\pi_k} = V^{\pi_{k+1}} = T^{\pi_{k+1}} V^{\pi_{k+1}} = T^{\pi_{k+1}} V^{\pi_k} = \mathcal{T}^\star V^{\pi_k}$$

so V^{π_k} is the fixed point of $\mathcal{T}^\star$: $V^{\pi_k} = V^\star$ and π_k is optimal.

OTHERWISE, IT STRICTLY IMPROVES, SO IT STOPS

If $V^{\pi_{k+1}} \neq V^{\pi_k}$, then $V^{\pi_{k+1}} \geq V^{\pi_k}$ with a strict inequality in some state. The sequence of value functions never repeats, and there are only $|\mathcal{A}|^{|\mathcal{S}|}$ deterministic policies. So policy iteration stops after finitely many steps, at an optimal policy. ■

In practice: break ties consistently (keep the current action if it is still greedy) and stop when the policy no longer changes.

SELF-STUDY

When can value iteration stop?

THEOREM (PUTERMAN 1994, THM 6.3.1)

If $\|V_{k+1} - V_k\|_\infty < \varepsilon \frac{1-\gamma}{2\gamma}$, and π is greedy w.r.t. V_{k+1}, then $\|V^\pi - V^\star\|_\infty \leq \varepsilon$: the policy is ε-**optimal**.

What it says

- A guarantee on the **loss of the policy** we output, measured on its true value V^π
- Computable: only uses two consecutive iterates

What it does not say

- That π is exactly optimal, or that the greedy policy will not change any more
- Finitely many policies, so a small enough ε gives exact optimality, but that threshold depends on unknown value gaps

IN THE MAZE ($\gamma = 0.9$, $\varepsilon = 0.1$, THRESHOLD ≈ 0.0056)

Deterministic: the rule stops at $k = 11$; the greedy policy was already optimal from $k = 8$.
Slippery: stops at $k = 20$; the greedy policy was optimal from $k = 13$. The rule is **conservative**.

SELF-STUDY

Proof of the stopping rule

Write $V' = V_{k+1} = \mathcal{T}^\star V_k$, $\Delta = \|V' - V_k\|_\infty$, and π greedy w.r.t. V', so $\mathcal{T}^\pi V' = \mathcal{T}^\star V'$. Split the error: $\|V^\pi - V^\star\| \le \|V^\pi - V'\| + \|V' - V^\star\|$.

(1) THE ESTIMATE IS CLOSE TO $V^\star$

$$\|V' - V^\star\| = \|\mathcal{T}^\star V_k - \mathcal{T}^\star V^\star\| \le \gamma \|V_k - V^\star\| \le \gamma\big(\Delta + \|V' - V^\star\|\big) \;\Rightarrow\; \|V' - V^\star\| \le \frac{\gamma}{1-\gamma}\Delta$$

(2) THE GREEDY POLICY'S VALUE IS CLOSE TO THE ESTIMATE

$$V^\pi - V' = \big(\mathcal{T}^\pi V^\pi - \mathcal{T}^\pi V'\big) + \big(\mathcal{T}^\star V' - \mathcal{T}^\star V_k\big), \quad \text{so } \|V^\pi - V'\| \le \gamma \|V^\pi - V'\| + \gamma\Delta$$
$$\Rightarrow\; \|V^\pi - V'\| \le \frac{\gamma}{1-\gamma}\Delta$$

Together: $\|V^\pi - V^\star\| \le \dfrac{2\gamma}{1-\gamma}\Delta < \varepsilon$ as soon as $\Delta < \varepsilon\dfrac{1-\gamma}{2\gamma}$. ■

Bonus from (1): the value estimate itself is then within $\varepsilon/2$ of $V^\star$.

References

- R. Sutton, A. Barto. *Reinforcement Learning: An Introduction*, 2nd ed., 2018. Chapters 3–4.
- M. Puterman. *Markov Decision Processes*, 1994. Chapter 6 (Thm 6.3.1: stopping rule for value iteration).
- Cs. Szepesvári. *Algorithms for Reinforcement Learning*, 2010.
- D. Silver. UCL RL course, Lectures 2–3.
- G. Martius, P. Kolev. RL Lecture 2, Universität Tübingen, 2026.
- A. Karpathy. REINFORCEjs GridWorld DP demo.